

The tally counter

Algebra, constructions, and bounds

How digit transformations and decimal carry can be used to plan a solution.

This document develops the counter puzzle from its rules. It explains why reset moves resemble matrix products, why increments behave differently, and how an empty-column construction gives a deterministic path to any target. It also proves increment bounds and shows how to turn them into search heuristics.

The main examples are deliberately small before the general arguments are introduced. Leading zeros are retained throughout: 012 is a three-wheel display, not a two-wheel display.

What is established

- Every start can reach every target.
- Four wheels require at most six button increments, with free knob turns.
- Reset moves have an exact linear representation on digit-indicator matrices.
- A backward column-packing algorithm gives a valid path without searching.
- Cheap lower bounds can guide search; some longer targets force large final stretches of increments.
- The sharp worst-case increment count grows exponentially with width.

Contents

1. The rules and the quantity being minimized	3
2. Reachability and a first direct construction	4
3. Two matrix encodings	5
4. Carry and the limits of Gaussian elimination	7
5. A deterministic column-packing construction	8
6. A cheap lower bound from weighted boundaries	11
7. Forced final increments	12
8. Bounds for the sharp worst case	14
9. Using the results in a solver	15

Reading route

Sections 1-2 define the rules and the simplest construction. Sections 3-5 explain the algebra and give the column-packing algorithm, with complete examples. Sections 6-8 develop lower bounds and worst-case growth. Section 9 explains how the results can be used in a solver.

1. The rules and the quantity being minimized

1.1. Fixed-width displays

A counter has $n \geq 1$ wheels. Its display is a digit vector $x = (x_0, x_1, \dots, x_{n-1})$, ordered from most significant to least significant, with each digit in $\mathcal{D} = \{0, 1, \dots, 9\}$. Its numeric value is

$$V(x) = \sum_{i=0}^{n-1} x_i 10^{n-1-i}, \quad M = 10^n.$$

For example, 012 has vector $(0, 1, 2)$ and value 12, but its width is three. An increment of 999 gives 000, because arithmetic wraps modulo $M = 1000$. We use d^n for the word containing n repetitions of digit d ; for example, 0^4 means the display 0000, rather than an ordinary numeric power.

The complete mechanical state also includes a knob index $r \in \{0, \dots, 9\}$. The displayed value does not determine this index. It specifies which digit will be engaged by the next forward knob tick.

1.2. Button increments and knob ticks

The button adds one with normal decimal carry:

$$V(\text{Inc}(x)) = (V(x) + 1) \bmod M.$$

A forward knob tick at index d replaces **every** occurrence of d by $(d + 1) \bmod 10$, and moves the knob index to that successor. We denote its display effect by R_d :

$$R_{d(x)_i} = \begin{cases} (d + 1) \bmod 10 & \text{if } x_i = d \\ x_i & \text{otherwise} \end{cases}$$

A backward tick changes only the knob index. It does not move any displayed wheel. By turning backward, we can position the knob at any chosen d without changing the display, then apply R_d with one forward tick.

Example: one digit value, all its occurrences

From 6060, the move R_6 gives 7070. It cannot give 7060, because the same digit map is applied to every wheel. From 0990, R_9 gives 0000: this knob move has no decimal carry.

1.3. Primary and secondary costs

Let $c(s, t)$ be the fewest button increments needed to reach target display t from start display s , allowing unlimited knob turns. Repositioning the knob is free for this primary cost, so $c(s, t)$ does not depend on the initial knob index.

The project additionally minimizes physical knob ticks among solutions with the fewest increments. That is a second objective. A sequence of ten selected digit maps R_d can require more than ten physical ticks, because the knob may need backward repositioning between them.

In this document, an increment bound concerns c , not total physical ticks. Constructed paths are valid for the complete mechanism, but are not claimed to minimize either cost unless an explicit proof says so.

1.4. Two basic facts about resets

Equal wheels remain equal. If $x_i = x_j$, any reset-only sequence gives the same output at positions i and j . A common digit map cannot split an equality class. It can merge previously different classes.

A forward tick leaves a digit missing. After R_d , no wheel shows d . Existing occurrences move to its successor, and no other digit moves into d . This will be important when a target contains all ten decimal digits.

An inverse reset used during mathematical planning is a **predecessor relation**. It is not the physical backward knob operation. We will explicitly construct predecessors that a legal forward tick maps to the current display.

2. Reachability and a first direct construction

2.1. A full revolution gathers the wheels

Start at knob index r and turn forward ten ticks. Each wheel is engaged when the knob reaches its original digit. Once engaged, it follows the knob through the remaining ticks. At the end every wheel shows r and the index is again r .

Continue by $\delta = (d - r) \bmod 10$ ticks. The display becomes uniformly d . Thus any start can reach any uniform display using at most $10 + 9 = 19$ forward ticks and zero button increments. In particular, any start can reach the zero display.

Example: an arbitrary starting state

If the knob is at index 3, a full forward revolution makes every wheel show 3. Five additional forward ticks make every wheel show 8. The starting display can have been anything.

2.2. Choose a nearby uniform display

Write

$$U_n = \underbrace{11\dots1}_{n \text{ digits}} = \frac{10^n - 1}{9}.$$

The ten uniform values are $0, U_n, 2U_n, \dots, 9U_n$. For a target of value T , choose

$$d = \left\lfloor \frac{T}{U_n} \right\rfloor, \quad k = T - dU_n.$$

First make the display uniformly d , then press the button k times. Since $0 \leq T \leq 9U_n$, the chosen d is a valid digit. Integer division gives $0 \leq k \leq U_n - 1$, including $k = 0$ for the all-nines target.

Example: reaching 9876

For four wheels, $U_4 = 1111$. We obtain $d = 8$ and $k = 9876 - 8 \cdot 1111 = 988$. Make the display 8888, then apply 988 increments. This is a guaranteed solution, even though a better one uses only six.

This construction proves

$$c(s, t) \leq \frac{10^n - 10}{9}.$$

Direct increments from the original start are another candidate, costing $D = (V(t) - V(s)) \bmod 10^n$. We can choose the cheaper of the two constructions.

2.3. A universal bound versus a particular puzzle

Define the sharp universal bound

$$W(n) = \max_{s, t} c(s, t).$$

Because a reset-only prefix takes every start to zero, $c(s, t) \leq c(0^n, t)$. Zero is itself an allowed start, so

$$W(n) = \max_t c(0^n, t).$$

This equality concerns the maximum over all starts. It does not say that every fixed start has the same hardest target or the same maximum. A particular start may already contain useful distinctions between its wheels. If start and target agree, their minimum is zero.

3. Two matrix encodings

The word “isomorphism” requires us to specify what structure is preserved. Numbers can be encoded by matrices in several ways. An encoding that makes addition simple need not also make reset moves simple.

3.1. An exact group isomorphism for modular addition

For $M = 10^n$, consider matrices over the ring $\frac{\mathbb{Z}}{M}\mathbb{Z}$:

$$G(T) = \begin{pmatrix} 1 & T \\ 0 & 1 \end{pmatrix}, \quad 0 \leq T < M.$$

Their product is

$$G(S)G(T) = \begin{pmatrix} 1 & S+T \\ 0 & 1 \end{pmatrix} = G((S+T) \bmod M).$$

The map $T \rightarrow G(T)$ is a group isomorphism from modular addition to this matrix family under multiplication. The identity is $G(0)$, and $G(T)^{-1} = G((-T) \bmod M)$. A button increment becomes multiplication by $G(1)$.

Example: carry is already included

At width three, $G(999)G(1) = G(0)$ because the upper-right entry is computed modulo 1000. This encoding handles carry through the whole numeric value.

It does not turn a reset into multiplication by a fixed member of this group. Such multiplication is bijective, whereas a reset can merge different states: for example, R_0 maps both 00 and 11 to 11. The reset effect also depends on which decimal digits occur in the value.

3.2. A digit-indicator matrix for resets

Instead encode the display by an $n \times 10$ matrix:

$$X(x)_{i,d} = \begin{cases} 1 & \text{if } x_i = d \\ 0 & \text{otherwise} \end{cases}$$

Columns are labeled 0, 1, ..., 9. Each row contains exactly one 1. This is often called a one-hot encoding: the location of that 1 records the digit.

For 507, the matrix is

$$X(507) = \begin{pmatrix} 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \end{pmatrix}.$$

The first row selects digit 5, the second selects 0, and the third selects 7. The representation is bijective: each display has one such matrix, and each such matrix decodes to one display.

The set of these matrices is not a vector space. Adding two valid matrices usually gives a row with two ones, or an entry equal to two. Thus we have a bijection with a restricted matrix set, not a linear-space isomorphism to all $n \times 10$ matrices.

To recover the numeric value, define

$$w = \begin{pmatrix} 10^{n-1} \\ 10^{n-2} \\ \dots \\ 1 \end{pmatrix}, \quad \ell = \begin{pmatrix} 0 \\ 1 \\ \dots \\ 9 \end{pmatrix}.$$

Then the ordinary matrix expression $V(x) = w^T X(x) \ell$ returns the displayed integer. A separate index r still records the physical knob position.

3.3. Reset matrices and their products

Let e_d be column d of the ten-dimensional identity matrix, and let $\sigma(d) = (d + 1) \bmod 10$. Define

$$A_d = I_{10} + e_d (e_{\sigma(d)} - e_d)^T.$$

Every row of A_d is the corresponding identity row, except row d , which selects the successor digit. Right multiplication therefore gives exactly

$$X(R_{d(x)}) = X(x) A_d.$$

Writing X_d for column d , this means

$$X'_d = 0, \quad X'_{\sigma(d)} = X_d + X_{\sigma(d)},$$

with every other column unchanged. The two columns have disjoint support, because no wheel initially has two digits. Their sum is still an indicator column, so the output remains a valid display matrix.

Example: merging classes

In 6060, columns 6 and 0 mark alternating wheels. Multiplication by A_6 moves the support of column 6 to column 7, producing 7070. In 6767, the same move merges columns 6 and 7, producing 7777.

A selected reset sequence is a product $X A_{d_1} A_{d_2} \dots A_{d_k}$. In general, order matters. From 01, applying R_0 then R_1 gives 22; applying R_1 then R_0 gives 12.

Each generator is idempotent:

$$A_d^2 = A_d.$$

After the first application there is no digit d to move, so a second application of the **same selected digit map** has no additional effect. This does not describe two consecutive physical forward ticks, which engage successive knob indices. It also does not imply that every product of the generators is idempotent. For example, $A_1 A_0$ sends a digit 0 to 1 on its first application, and that 1 to 2 on its second application. Thus $(A_1 A_0)^2 \neq A_1 A_0$.

Each A_d has rank 9: its rows select every basis direction except d , with the successor direction selected twice. Hence it is singular, not a permutation matrix. These generators and their products form a **transformation monoid**: a collection closed under composition, with an identity, but without an inverse for every element.

The rank of X equals the number of distinct displayed digits. Its nonzero columns have disjoint support and are linearly independent. Resets cannot increase that rank. They preserve it when moving a class into an empty column and reduce it when merging two occupied columns.

4. Carry and the limits of Gaussian elimination

4.1. The increment as a state-dependent row operation

Let C be the 10×10 cyclic permutation matrix satisfying $e_d^T C = e_{\sigma(d)}^T$. It rotates a digit through $0 \rightarrow 1 \rightarrow \dots \rightarrow 9 \rightarrow 0$.

An increment rotates only wheels reached by the carry. Define a diagonal mask $Q(X)$ by

$$Q(X)_{i,i} = \prod_{j=i+1}^{n-1} X_{j,9}.$$

An empty product is 1, so the last wheel always changes. A more significant wheel changes exactly when every wheel to its right shows 9. Therefore

$$X(\text{Inc}(x)) = (I_n - Q(X))X + Q(X)XC.$$

Rows whose mask entry is zero remain unchanged. Rows whose mask entry is one are multiplied by the digit-cycle matrix.

Example: 1299 becomes 1300

The mask is $Q = \text{diag}(0, 1, 1, 1)$. The leading 1 stays fixed. The 2 becomes 3, and both nines become zeros. For 9999, every mask entry is 1, so the formula gives 0000.

For a fixed mask this is a linear operation on the matrix entries. The actual mask is computed from the entries of X , however, so the full increment map is state-dependent and nonlinear in this encoding.

4.2. Why ordinary elimination does not immediately solve the puzzle

Gaussian elimination solves linear systems using permitted invertible row operations. The counter has different constraints:

- Reset operations are restricted global column maps. We cannot independently modify one selected wheel showing a digit.
- Reset matrices can destroy information by merging columns, so their general inverses do not exist.
- The carry mask for an increment depends on the current display.
- Intermediate matrices must remain valid one-hot display matrices, and every step must correspond to a legal counter operation.

Knowing a matrix equation for the desired endpoint does not ensure that an arbitrary factorization of that equation uses legal operations. The problem is a constrained action-sequence problem, rather than an ordinary linear system.

There is a much larger linear encoding: use one basis vector for each of the 10^n displays. Increment is then a fixed cyclic permutation matrix, and each reset is a fixed deterministic transition matrix. Both operations become linear, but the dimension is exponential, and selecting a legal word in the generators remains the original planning problem.

The useful analogy

An empty digit column can act like a pivot location. We can use it to undo selected reset maps during backward planning. This leads to a direct construction, without assuming that arbitrary matrix elimination is legal.

5. A deterministic column-packing construction

5.1. A legal predecessor through an empty column

Suppose digit $a - 1$ is absent from the current display and digit a occurs, with $1 \leq a \leq 9$. Move every occurrence of a to $a - 1$ during planning. Call the resulting predecessor y . Then

$$R_{a-1}(y) = x.$$

Indeed, the new occurrences of $a - 1$ are precisely the wheels that originally showed a . No other wheels originally showed $a - 1$. The forward map restores exactly those wheels and leaves the others alone.

In matrix language, transfer the entire occupied column a into empty column $a - 1$. This is an inverse on the particular state being considered, not an inverse of the singular matrix A_{a-1} on every matrix.

Example: 020 has a predecessor 010

Digit 1 is absent from 020. Moving the 2 to 1 gives 010, and the legal forward move R_1 maps 010 back to 020. If the current display were 120, this particular predecessor operation would fail: R_1 would also move the wheel that originally showed 1.

5.2. Packing all occupied columns

Let the distinct current digits, in increasing order, be $a_0 < a_1 < \dots < a_{q-1}$. Move class a_j down to digit j , processing the classes from smallest to largest. The final occupied columns are $0, 1, \dots, q - 1$.

Why are the intermediate destinations empty? Earlier processed classes occupy only digits below j . Later unprocessed classes are larger than the current class. Since a_j is the next occupied column in sorted order, every position between j and $a_j - 1$ is empty. We can therefore move one column left at a time using legal predecessors.

Example: packing 9876

Its distinct digits are 6, 7, 8, 9. Move the 6-class to 0, the 7-class to 1, the 8-class to 2, and the 9-class to 3. The wheel order stays fixed, so the display becomes 3210, not 0123. These are 24 predecessor transfers: six for each class.

A pass uses $\sum_{j=0}^{q-1} (a_j - j)$ transfers. Because $a_j \leq 10 - q + j$, this is at most $q(10 - q) \leq 25$. This bounds selected digit transfers per packing pass, not physical knob ticks for the eventual replay. The inequality follows because $q - 1 - j$ larger occupied digits must still fit between $a_j + 1$ and 9.

5.3. The complete backward algorithm

Starting from the target:

1. Pack its occupied columns as above and record the corresponding forward reset maps.
2. If the display is all zeros, stop.
3. Otherwise subtract one with normal decimal borrow and record an increment.
4. Repeat.
5. Reverse the recorded list and execute it from the zero display.

In step 1 the recorded forward map is R_{a-1} for each predecessor transfer $a \rightarrow a - 1$. Step 3 records the forward increment that reverses the subtraction. Reversing the **entire order** is essential.

```

display := target
recorded := empty list

repeat:
    occupied := sorted distinct digits in display

    for each (rank, original_digit) in occupied:
        digit := original_digit

        while digit > rank:
            assert digit - 1 is absent from display
            replace every occurrence of digit with digit - 1
            record the forward map  $R_-(digit - 1)$ 
            digit := digit - 1

    if display is all zeros:
        stop

    display := display - 1, retaining its width
    record a forward increment

return reverse(recorded)

```

For an arbitrary starting display, first use legal knob turns to reach zero, then execute the constructed sequence.

5.4. Proof of validity and termination

Each packing transfer has the legal forward inverse proved above. Each subtraction has a legal forward increment as its inverse, since subtraction is used only at a positive value. Thus reversing all recorded steps gives an exact path from zero to the original target.

Every packing transfer strictly decreases the numeric value: it decreases a positive digit on at least one wheel and increases no digit. Every subtraction also decreases the value by one. The value is a nonnegative integer, so there cannot be infinitely many steps. If it is positive after packing, the algorithm performs a subtraction and continues. It therefore terminates at zero.

If the initial target value is T , the construction uses at most T increments. After the first packing pass, the remaining numeric value gives a usually smaller such upper bound. These guarantees are not polynomial bounds in the number of wheels, because T can be of order 10^n .

Reversing the construction gives a monotonically increasing numeric path. Its selected reset maps use only $R_0, \dots, R_8$, so those moves do not wrap a digit from 9 to 0. Decimal carry can still turn trailing nines to zeros during an increment, while the whole numeric value increases by one.

5.5. A complete small example: target 121

The backward plan is:

Current	Predecessor	Reason and recorded forward action
121	020	Digit 0 is absent. Move all 1s to 0; record R_0 .
020	010	Digit 1 is absent. Move all 2s to 1; record R_1 .
010	009	Subtract one; record an increment.
009	001	Move 9 down through empty columns 8 to 1; record $R_8, R_7, \dots, R_1$.
001	000	Subtract one; record an increment.

Reverse those actions to obtain the physical forward plan:

$$000 \xrightarrow{+1} 001 \xrightarrow{R_1, \dots, R_8} 009 \xrightarrow{+1} 010 \xrightarrow{R_1} 020 \xrightarrow{R_0} 121.$$

The eight maps R_1 through R_8 raise only the last wheel from 1 to 9. Incrementing 009 then creates 010. Finally R_1 gives 020, and R_0 moves both zeros, giving the desired 121.

There are two button increments and ten selected reset maps. Starting with knob index zero, this particular plan can be executed with 29 physical knob ticks: nine backward ticks to select the first R_1 , eight forward ticks through R_8 , eight backward ticks to select the later R_1 , its forward tick, then two backward ticks and one forward tick for R_0 .

The target has two unequal neighboring digits, and the lower-bound argument in Section 6 gives two increments. Thus this example is increment-optimal, although this particular knob sequence is not claimed to minimize reset ticks.

5.6. A larger example: target 9876

Initial packing changes 9876 to 3210. In the table below, each row performs one backward subtraction and then packs the columns. A row whose last two entries agree needs no packing transfers.

Step	Before subtraction	After subtraction	After packing
1	3210	3209	2103
2	2103	2102	2102
3	2102	2101	2101
4	2101	2100	2100
5	2100	2099	1022
6	1022	1021	1021
7	1021	1020	1020
8	1020	1019	1012
9	1012	1011	1011
10	1011	1010	1010
11	1010	1009	1002
12	1002	1001	1001
13	1001	1000	1000
14	1000	0999	0111
15	0111	0110	0110
16	0110	0109	0102
17	0102	0101	0101
18	0101	0100	0100
19	0100	0099	0011
20	0011	0010	0010
21	0010	0009	0001
22	0001	0000	0000

Reversing this trace gives a path using 22 increments and 85 selected reset maps, including the inverse of the initial packing pass. The uniform-display construction uses 988 increments, while exhaustive optimal search uses six. The packing algorithm is a direct construction with a feasible endpoint, not an optimality theorem.

6. A cheap lower bound from weighted boundaries

6.1. Why unequal neighbors reveal necessary work

Starting from a uniform display, reset moves cannot create unequal neighbors. Increments must create those distinctions. A boundary farther left is harder to create because the carry must reach farther into the counter.

For each boundary between positions i and $i + 1$, give it weight $w_i = n - i - 1$. The available boundaries are those with $x_i \neq x_{i+1}$. Select available boundaries without selecting two that share a wheel, and maximize their total weight. Call the resulting score $B(x)$:

$$B(x) = \max_A \sum_{i \in A} (n - i - 1),$$

where A contains only available boundaries and no two of its indices are consecutive. The empty set is permitted, with score zero.

Example: why adjacent boundaries are not all added

0123 has available weights 3, 2, 1. Adding all of them gives 6, which is not a valid lower bound: this target needs only four increments. Adjacent boundaries can share work. Selecting the first and last, which do not share a wheel, gives the valid score $B = 4$.

6.2. A one-pass recurrence

Let P_i be the best score from boundary i onward, and set $P_{n-1} = P_n = 0$. Work from right to left:

$$P_i = \begin{cases} \max(P_{i+1}, n - i - 1 + P_{i+2}) & \text{if } x_i \neq x_{i+1}, \\ P_{i+1} & \text{otherwise} \end{cases}, \quad B(x) = P_0.$$

The first choice skips the boundary. The second takes it and skips the next boundary, which shares a wheel. Each position is processed once, and only the next two scores need to be stored: time is $O(n)$ and auxiliary score storage is constant.

For 9876, the calculation is $P_3 = P_4 = 0$, then $P_2 = 1$, $P_1 = \max(1, 2) = 2$, and $P_0 = \max(2, 3 + 1) = 4$.

6.3. Proof that the score is a potential

Resets cannot increase B . An equal neighboring pair stays equal under a common digit map. Thus a reset can remove available boundaries but cannot add one. Every boundary selection available after it was available before.

An increment increases B by at most one. There are three cases:

1. With no carry, only the last wheel changes. Only the last boundary can change its availability, and its weight is one.
2. With a nonempty suffix of nines, let p be the rightmost non-nine digit. The suffix changes from all nines to all zeros. Its internal boundaries remain equal. The boundary just after p is unequal before and after: before it is between a non-nine and a nine; after it is between a nonzero digit and zero. The only possible new boundary is just before p .
3. If every digit is nine, the display becomes all zeros; both scores are zero.

In case 2, suppose a best new selection uses the new boundary just before p . Replace it by the old boundary just after p . The replacement is compatible with the other selected boundaries: the edge to its left was already excluded, and no unequal edges occur farther right inside the uniform suffix. The weight drops by exactly one:

$$(n - p) - (n - p - 1) = 1.$$

The old display therefore had a legal selection worth at least the new score minus one. If the new selection does not use that new boundary, it was already available before. This proves the claim in every case.

Consequently, any path with k increments satisfies

$$B(t) \leq B(s) + k, \quad h_{B(s,t)} = \max(0, B(t) - B(s)) \leq c(s, t).$$

This is an **admissible** lower bound: it never overestimates the minimum. It holds for arbitrary starts, not only zero. From any uniform start, $B(s) = 0$.

Target from zeros	Lower bound	Exact increments
1000	3	3
0101	4	4
0123	4	4
9876	4	6
121	2	2

6.4. Consistency for search

For a fixed target, the heuristic $h_{B(x,t)}$ satisfies $h_{B(x,t)} \leq w(a) + h_{B(a(x),t)}$ for each action a , where $w(a)$ is one for an increment and zero for a knob move. This follows directly from the potential inequalities. Such a heuristic is called **consistent**.

In reverse search toward a fixed initial display s , use $h_{B(s,x)}$ instead. The same inequality holds on reverse edges. Consistency is useful because the estimated total increment cost does not decrease along a concrete search path. It does not by itself justify treating a whole group of different displays as one concrete state.

7. Forced final increments

7.1. Complete displays cannot be entered by a forward reset

Call a display **complete** when it contains all ten decimal digits. As proved in Section 1, every output of R_d lacks digit d . Therefore a complete display has no incoming forward-reset transition. Reading a successful path backward, its only display-changing predecessor is one decrement.

Backward knob ticks may occur physically, but they do not change the display. They do not provide an alternative predecessor value.

7.2. Find the beginning of the forced interval

For target value T , let

$$U = \max\{v : 0 \leq v \leq T, \text{Missing}(\text{digits}_{n(v)})\},$$

$$u = \text{digits}_{n(U)}, \quad H(t) = T - U.$$

Here “Missing” means that at least one decimal digit is absent, counting leading zeros. The set always contains zero. If the target is already missing a digit, then $u = t$ and $H(t) = 0$. Otherwise, every display at values $U + 1, \dots, T$ is complete. Entering them forces the last $H(t)$ increments.

Example: a forced interval of length 111

For target 012345678999, the largest preceding display missing a digit is 012345678888. Its prefix supplies digits 0 through 8. Every suffix from 889 through 999 contains a 9, whereas suffix 888 does not. Thus $H = 999 - 888 = 111$.

7.3. Compute the predecessor without walking through the interval

For each digit d , find the largest same-width display at or below the target that avoids d . Then take the largest candidate over all ten digits.

If d is absent already, the target itself is a candidate. Otherwise:

1. Preserve the target prefix up to its first occurrence of d .
2. At that position, choose the largest smaller allowed digit. If no such choice exists, move left to the nearest earlier position that can be lowered to an allowed digit.
3. Fill the remaining suffix with the largest allowed digit: 9 normally, or 8 when the excluded digit is 9.
4. If no position can be lowered legally, discard this excluded-digit case.

Keeping the longest possible prefix maximizes the candidate. The smallest permitted decrease at the next position, followed by the largest suffix, maximizes it among candidates with that prefix. Every incomplete display avoids at least one digit, so the maximum over these cases is exactly u .

This takes ten linear digit scans, or $O(10n) = O(n)$ digit operations. The predecessor and the difference H can be computed using digit strings, without requiring the whole displayed value to fit a machine integer.

Example: excluding zero may need a borrow

For 1203456789, the first zero cannot be lowered. Move left to the 2, lower it to 1, and fill the suffix with nines: 119999999 is the largest predecessor avoiding zero. For a target beginning with zero, there may be no zero-free predecessor below it at that same width; that case is skipped.

7.4. Arbitrary starts and an exact decomposition

The forced-tail bound must account for a start already inside the interval. Let $D = (T - V(s)) \bmod M$. Then

$$c(s, t) = \min(D, H(t) + c(s, u)).$$

Reasoning. Direct increments give a path costing D . Reaching u and appending H increments gives a path costing $H + c(s, u)$. Conversely, a successful path either starts inside the final forced interval and approaches the target without first reaching u , or must pass through u before its final approach. In the first case it needs at least D increments; in the second it needs at least $c(s, u) + H$. No display-changing reset can enter an interior point of the interval. These lower and upper arguments give the equality.

If $U \leq V(s) \leq T$, the exact increment optimum is $T - V(s)$, attained without knob turns. If the start is outside that interval, every solution must pass through u , and the exact optimum is $c(s, u) + H$. A wraparound earlier in a path does not bypass the final interval.

For the project's secondary reset objective, solve the prefix to u with its final knob index unconstrained, then append H increments. They preserve that index. Unnecessary backward knob ticks in the tail can be omitted because the final index is unconstrained, so this reduction also preserves the best reset cost.

7.5. Combine the forced interval with the cheap bound

Substitute a lower bound for $c(s, u)$ in the exact decomposition:

$$h(s, t) = \min(D, H(t) + \max(0, B(u) - B(s))) \leq c(s, t).$$

If the target lacks a digit, $H = 0$ and this reduces to the boundary bound: $h_{B(s,t)} \leq D$ because direct increments are a legal path.

For the twelve-wheel example, $B(0^{12}) = 0$ and $B(012345678888) = 32$. The available boundary weights for this predecessor are 11, 10, ..., 4; selecting 11, 9, 7, 5 gives 32. Therefore

$$c(0^{12}, 012345678999) \geq 111 + 32 = 143.$$

This is stronger than the forced-tail bound 111 alone. It is still a lower bound, not an assertion that 143 increments suffice.

During backward construction or search, a complete display's forced interval can also be replaced by one grouped decrement of size H . All intermediate states have all ten digits and therefore offer no empty-column predecessor transfers. This skips their explicit enumeration while preserving their button cost. If the specified initial display lies inside the interval, the search must stop there rather than jump past it.

8. Bounds for the sharp worst case

8.1. Exact small cases

The quotient state graph has one vertex per display. Its zero-cost edges are the ten selected digit maps R_d , and its unit-cost edge is an increment. Free backward knob positioning makes every selected map available. A zero-one breadth-first search from zero computes every target's minimum increment count.

Exhaustive computation gives:

Width	Sharp bound $W(n)$	A hardest target
1	0	0
2	1	01
3	3	021
4	6	0132 or 9876
5	11	04321
6	17	054321
7	25	0654321
8	34	07654321

For four wheels, the counts of targets requiring exactly 0, 1, 2, 3, 4, 5, 6 increments are respectively 10, 90, 540, 900, 2460, 4320, 1680. They total 10,000. This establishes six as both sufficient for every target from zero and necessary for some targets.

The reset-only prefix argument then extends the six-increment upper bound to every four-wheel start. The computation ignores secondary reset cost; it does not establish the fewest physical knob ticks.

8.2. A quadratic lower bound

Choose a target with every neighboring pair unequal, such as 010101... Every boundary is available to the potential B . Its best selection takes alternating boundary weights.

For $n = 2m$, the weights selected are $2m - 1, 2m - 3, \dots, 1$, summing to m^2 . For $n = 2m + 1$, they are $2m, 2m - 2, \dots, 2$, summing to $m(m + 1)$. Earlier available weights can always replace later ones without reducing the sum, so these alternating choices are optimal. Hence

$$W(n) \geq \left\lfloor \frac{n^2}{4} \right\rfloor.$$

This is a statement about the hardest possible target, not a lower bound for every target of width n . A uniform target always needs zero increments.

8.3. An exponential lower bound

For $n \geq 10$, put $m = n - 9$ and choose the target

$$t = 012345678 \underbrace{99\dots9}_m \text{ nines}.$$

The fixed prefix contains digits 0 through 8. The suffix starts at $10^m - 1$. The largest m -digit suffix without any nine is the all-eights suffix, with value $8 \frac{10^m - 1}{9}$.

Until the suffix reaches that value during backward decrements, it still contains a nine. All ten digits therefore remain present, forcing every backward display change to be a decrement. The compulsory count is

$$(10^m - 1) - 8 \frac{10^m - 1}{9} = \frac{10^m - 1}{9}.$$

Therefore

$$W(n) \geq \frac{10^{n-9} - 1}{9} \quad \text{for } n \geq 10.$$

Example: why the small values do not imply a quadratic law

At width 12, target 012345678999 already forces 111 final increments. The formula $\frac{n(n-1)}{2} + \left\lfloor \frac{(n-3)^2}{4} \right\rfloor$ happens to fit the computed widths 2 through 8, but predicts only 86 at width 12. It cannot be the general formula. The combined heuristic raises this particular target's lower bound further, to 143.

Combining the established results gives

$$\left\lfloor \frac{n^2}{4} \right\rfloor \leq W(n) \leq \frac{10^n - 10}{9},$$

and, for $n \geq 10$,

$$\max \left(\left\lfloor \frac{n^2}{4} \right\rfloor, \frac{10^{n-9} - 1}{9} \right) \leq W(n) \leq \frac{10^n - 10}{9}.$$

In particular, $10^{n-10} \leq W(n) < \frac{10^n}{9}$ for $n \geq 10$. The fixed constants do not depend on n , so

$$W(n) = \Theta(10^n).$$

This determines the asymptotic growth rate, not an exact closed formula for all widths. It also concerns the number of button presses in a solution, not how much computation is required to describe a grouped solution.

9. Using the results in a solver

9.1. Upper bounds, lower bounds, and exact answers

The uniform construction and column packing produce feasible solutions. Their increment counts are upper bounds. A solver can use the cheapest known feasible path as an incumbent.

The potential and forced-tail arguments produce lower bounds. They say that no legal solution can use fewer increments than the bound. They need not describe a solution attaining it.

If a feasible construction uses exactly as many increments as a proved lower bound, its increment count is optimal. This still does not prove that its knob tick count is the best among increment-optimal paths.

Example: the three possible outcomes

For 121 from zeros, the packing path uses two increments and the lower bound is two: increment optimality is certified. For 9876, the cheap lower bound is four and packing uses 22: there is a gap, so the optimum is not certified by those two numbers. Exact search closes the gap at six.

9.2. A* priorities and symbolic groups

For a concrete forward search, prioritize a state x using its increments already spent plus $h(x, t)$. For a concrete reverse search from the target toward the initial display s , use the reverse cost already spent plus $h(s, x)$. The orientation matters: the heuristic estimates the part of the path not yet connected.

The project's exact solver groups many displays into decision-diagram regions. A heuristic used for a group $\mathcal{G}$ must be valid for every represented display. The rigorous group value is

$$h(\mathcal{G}) = \min_{x \in \mathcal{G}} h(s, x).$$

Using a score from one representative, or the largest score in the group, could overestimate the remaining cost for another member. A group-level minimum can also be weaker than the individual scores. The interaction with settling, merging, reconstruction, and the secondary reset objective must be verified when changing the search priority.

9.3. Suggested deterministic workflow

1. If start and target already agree, return no operations.
2. Find any mandatory final interval and remove it from planning, handling a start inside the interval directly.
3. Construct an initial feasible path using uniform displays or column packing.
4. Compute a lower bound with the boundary potential, strengthening it with the mandatory interval.
5. If the feasible increment count meets the lower bound, its primary cost is certified. Otherwise retain it as an upper bound for exact search.
6. Append any removed final increments and replay the resulting physical path.

This separates three tasks: constructing a path, proving a cost bound, and searching for an optimum. The matrix representation is particularly useful for understanding reset structure and legal predecessors; it does not make these three tasks identical.

9.4. Verification and scope of the conclusions

The proofs of reset behavior, matrix formulas, packing termination, boundary potential, forced intervals, and asymptotic bounds apply to every width under the stated model.

Finite checks additionally verified the packing construction by complete forward replay for all 1,000 three-wheel displays, 200 sampled four-wheel displays, and 50 sampled five-wheel displays. The 22-increment packing path for 9876 was reproduced independently. The exact small-case table came from exhaustive zero-one search, and the boundary inequalities were checked against every four-wheel increment and selected reset transition.

Those checks support the calculations; they are not substitutes for the arbitrary-width proofs. The sharp value of $W(n)$ for every n has not been derived here. Neither the packing construction nor these increment bounds provide a general polynomial-time optimal solver.

The accompanying repository defines the physical operations in `src/counter.rs`, the boundary and forced-predecessor calculations in `src/search/bounds.rs`, and exact search in `src/search.rs`. Its longer `docs/mathematics-and-solver.md` also describes implementation details and progress estimates. This document focuses on the mathematical discussion and direct construction.

The algebraic picture

Modular addition has a small matrix group model. Reset digit maps have a useful one-hot matrix monoid model. Carry couples the wheels in a state-dependent way. Empty columns let us construct legal predecessors, and reversing a decreasing predecessor sequence gives a direct path to the target.